

DevOps 구현

1. DBMS 1

1.1 DevOps 활용

- **데브옵스(DevOps)** : 소프트웨어의 개발(Development)과 운영(Operations)의 합성어
- **핵심 요소**
 - CI/CD (지속적 통합 및 지속적 배포): 자동화된 빌드, 테스트, 배포
 - IaC (Infrastructure as Code): 코드로 인프라 관리
 - 인프라 : 시스템 구조 및 체계
 - 컨테이너 (Docker, Kubernetes): 애플리케이션 격리 및 배포
 - 모니터링 : 시스템 상태 감시 및 분석

1.2 SQL 환경설정

- **DBMS** : 데이터베이스 관리 시스템(Database Management System)
- **대표적인 DBMS** : MySQL, Oracle, PostgreSQL, SQLite, MongoDB
- **관계형 데이터베이스 (RDBMS)** : 테이블 기반 (예: MySQL, Oracle, PostgreSQL, SQLite)
- **비관계형 데이터베이스 (NoSQL)** : 문서, 키-값, 컬럼 기반 (예: MongoDB, Redis)
- **MYSQL** : 관계형 데이터베이스 관리 시스템(RDBMS)

1.3 SQL 개요

- **SQL** : 구조화된 질의 언어(Structured Query Language)
- **DDL (Data Definition Language)** : 데이터 정의 언어, 데이터베이스의 구조 관련 작업 수행
- **DML (Data Manipulation Language)** : 데이터 조작 언어, 데이터 조작을 위한 조회, 삽입, 수정, 삭제 작업 수행

- **DCL (Data Control Language)** : 데이터 제어 언어, 데이터베이스의 보안 및 권한 관리와 관련된 작업 수행
- **TCL (Transaction Control Language)** : 트랜잭션 제어 언어, 트랜잭션의 변경 사항의 영구적 저장 및 데이터베이스에 적용 작업 수행, DCL로 분류하는 경우도 있음

1.4 데이터베이스 객체

- **주요 객체**
 - 데이터베이스(Database)
 - 데이터를 논리적으로 그룹화하는 컨테이너 역할을 수행
 - 여러 개의 테이블, 뷰, 프로시저 등과 같은 객체들을 포함 가능
 - 테이블(Table): 데이터를 저장하는 기본 구조
 - 뷰(View): 가상 테이블로 특정 데이터를 쉽게 조회할 수 있음
 - 인덱스(Index): 검색 속도를 향상시키는 데이터 구조
 - 제약조건(Constraints): 데이터가 특정 조건을 만족하도록 강제
 - 함수(Functions): 저장된 프로그램으로, 특정 작업을 수행하고 값을 반환

2. DBMS 2

2.1 SQL 문장

- **DML**
 - SELECT : 테이블이나 뷰에 있는 데이터를 조회

```
SELECT 컬럼명 FROM 테이블명 [WHERE 조건] [ORDER BY 컬럼명];
SELECT name, salary FROM employees WHERE salary > 6000;
```

- INSERT : 데이터를 신규로 생성

```
INSERT INTO 테이블명 (컬럼1, 컬럼2, ...) VALUES (값1, 값2, ...);
INSERT INTO employees (name, department_name, salary) VALUES
('김철수', '영업팀', 5000);
```

- UPDATE : 기 생성된 데이터를 수정

```
UPDATE 테이블명 SET 컬럼1 = 변경값1, 컬럼2 = 변경값2, ... [WHERE 조건];  
UPDATE employees SET department_name = '연구소', salary = 8000  
WHERE employee_id = 3;
```

- DELETE : 데이터 삭제

```
DELETE [FROM] 테이블명 [WHERE delete조건];  
DELETE FROM employees WHERE salary >= 7000;
```

• TCL

- COMMIT : 변경된 데이터를 최종 적용

```
# AUTOCOMMIT 설정 및 해제  
SET AUTOCOMMIT = 1; -- 설정 (기본 설정)  
SET AUTOCOMMIT = 0; -- 해제 (세션 단위, 일시적)
```

- ROLLBACK : 변경된 데이터를 적용하지 않고 이전 상태로 되돌림

```
ROLLBACK;
```

- SAVEPOINT : 특정 시점으로 롤백할 수 있도록 저장점을 설정

```
SAVEPOINT 저장점_이름;
```

2.2 조인

- 조인(JOIN): 테이블 간의 관계를 맺는 방법
- INNER JOIN: 두 테이블에서 일치하는 데이터만 반환

```
SELECT A.컬럼명, B.컬럼명 FROM 테이블A AS A INNER JOIN 테이블B AS B  
ON A.공통컬럼 = B.공통컬럼;
```

```
SELECT Employees.Name, Departments.DeptName FROM Employees I  
NNER JOIN Departments ON Employees.DeptID = Departments.DeptID;
```

- LEFT JOIN (LEFT OUTER JOIN): 왼쪽 테이블의 모든 행과 일치하는 오른쪽 테이블의 데이터를 반환

```
SELECT A.컬럼명, B.컬럼명 FROM 테이블A AS A LEFT JOIN 테이블B AS B
ON A.공통컬럼 = B.공통컬럼;
```

```
SELECT Employees.Name, Departments.DeptName FROM Employees L
EFT JOIN Departments ON Employees.DeptID = Departments.DeptID;
```

- RIGHT JOIN (RIGHT OUTER JOIN): 오른쪽 테이블의 모든 행과 일치하는 왼쪽 테이블의 데이터를 반환

```
SELECT A.컬럼명, B.컬럼명 FROM 테이블A AS A RIGHT JOIN 테이블B AS B
ON A.공통컬럼 = B.공통컬럼;
```

```
SELECT Employees.Name, Departments.DeptName FROM Employees R
IGHT JOIN Departments ON Employees.DeptID = Departments.DeptID;
```

- FULL JOIN (FULL OUTER JOIN): 두 테이블의 모든 행을 반환 (MySQL에서는 지원하지 않으며 UNION을 사용함)
- CROSS JOIN: 두 테이블의 모든 행을 조합한 결과를 반환

```
SELECT A.컬럼명, B.컬럼명 FROM 테이블A AS A CROSS JOIN 테이블B AS
B;
```

```
SELECT e.ID, e.Name, e.DeptID, d.DeptID, d.DeptName FROM Employee
es e CROSS JOIN Departments d;
```

- SELF JOIN: 같은 테이블을 자기 자신과 조인

```
SELECT A.컬럼명, B.컬럼명 FROM 테이블명 AS A JOIN 테이블명 AS B ON
A.공통컬럼 = B.공통컬럼;
```

```
SELECT E1.Name AS Employee, E2.Name AS Manager FROM Employee
s AS E1 JOIN Employees AS E2 ON E1.ManagerID = E2.ID;
```

2.3 파이썬 DB 연동 (1)

```
# MySQL 연결 및 데이터베이스 선택
# MySQL 서버에 연결
conn = pymysql.connect( # Connection class 객체 생성
    host="localhost", # MySQL 서버 주소 (로컬인 경우 'localhost')
    user="root", # MySQL 사용자명
    password="password", # MySQL 비밀번호
    database="exampledb", # 사용할 데이터베이스 명
    charset='utf8mb4', # UTF-8의 확장 버전
    # DictCursor 클래스: sql 구문 실행 후, 조회된 결과를 딕셔너리 형태로 반환
    cursorclass=pymysql.cursors.DictCursor
)
cursor = conn.cursor() # cursor 함수 객체 생성
# 현재 선택된 데이터베이스 확인
cursor.execute("SELECT DATABASE()")
print("현재 데이터베이스:", cursor.fetchone())
conn.close()
```

2.4 파이썬 DB 연동 (2)

```
# 데이터 삽입
conn = pymysql.connect(host="localhost", user="root", password="1234",
    database="exampledb")
cursor = conn.cursor()
cursor.execute("INSERT INTO users (name, age, email) VALUES ('Alice', 25,
    'alice@example.com');")
conn.commit() # 변경 사항 저장
conn.close()

# 데이터 조회
conn = pymysql.connect(host="localhost", user="root", password="1234",
    database="exampledb")
```

```

cursor = conn.cursor()
cursor.execute("SELECT * FROM users")
users = cursor.fetchall()
for user in users:
    print(user)
conn.close()

# 데이터 수정
conn = pymysql.connect(host="localhost", user="root", password="1234",
database="exampledb")
cursor = conn.cursor()
# 사용자 정보 수정 (이름 변경)
cursor.execute("UPDATE users SET name = %s WHERE id = %s", ("Bob",
1))
conn.commit()
conn.close()

# 데이터 삭제
conn = pymysql.connect(host="localhost", user="root", password="1234",
database="exampledb")
cursor = conn.cursor()
# 특정 사용자 삭제
cursor.execute("DELETE FROM users WHERE id = 3")
conn.commit()
conn.close()

```

3. GIT 형상관리

3.1 GIT 개요

- GIT 용어
 - 로컬 저장소: 개인 작업 공간
 - 원격 저장소: 공용 작업 공간
 - origin : 대표적인 원격저장소 별칭
 - Push: 원격 저장소로 저장
 - Pull: 원격저장소에서 로컬저장소로 당겨옴

- Fetch: 원격저장소의 변경된 커밋만 ㄱ 로드
- Commit: 새로운 버전을 저장소에 저장
- Staging: 커밋을 위한 내용을 준비
- Merge: 다른 브랜치의 내용을 합치는 작업
- Checkout: 이전 버전을 복구, 또는 특정 커밋/브랜치로 이동
- Branch: 분기점. 현재 상태를 복사하여 별도의 작업을 수행
- master or main: 브랜치들 중 가장 기본적인 브랜치명
- HEAD: 현재 작업 위치 표시
- reset : 이전 버전 복구, 커밋 히스토리도 삭제, 로컬 저장소 사용
- revert : 이전 버전 복구, 커밋 히스토리는 보존

3.2 VSCode 연동

- Clone : .git 폴더까지 포함 소스 복제
- Pull : 원격저장소 파일로 로컬저장소를 갱신, 병합(merge) 발생

3.3 명령어 기반 GIT 활용

```
# 깃 초기화
git init
# 비어있는 깃 저장소 초기화 : 안전한 디렉터리 추가
git config --global --add safe.directory F:/rokey3
# 깃 상태 확인
git status
# 깃허브(원격 저장소) 계정 정보(아이디/이메일) 설정
git config --global user.name "깃허브아이디"
git config --global user.email "깃허브이메일"
# 스테이징 : Untracked files 통해 commit 가능 여부 확인
git add 폴더/파일명
# 스테이징 파일 제외
git restore --staged ch202/test303.py
# 커밋
```

```
git commit -m "커밋메세지"
# 커밋 이력 확인
git log
# 원격 저장소 구성
git remote add <원격저장소명> <원격저장소url>
# 원격 저장소 목록 확인
git remote -v
# 원격 저장소 연동 해제
git remote remove <원격저장소명>
# Push
git push [remote] [branch]
# Pull
git pull <원격저장소명> <브랜치명>
# Checkout
git checkout <커밋ID>
```

3.4 버전 관리

- 팀 프로젝트를 위한 협업 수행 예시
 - 담당 업무(기능) 분배
 - 원격 저장소 생성 : Github
 - 원격 저장소 등록
 - 로컬 저장소 생성 : 개인 PC
 - 기능 구현 완료 및 병합

4. Docker 활용

4.1 Docker 개요

- 도커(Docker)란?
 - 가상화 기술을 활용한 컨테이너화 아키텍처
 - 애플리케이션 개발과 배포를 효율적으로 수행
 - 개발자와 운영자의 작업 간소화 및 협업 가능
 - 도커는 가장 널리 사용되는 컨테이너 기술로 큰 커뮤니티와 생태계를 보유

- **컨테이너(Container)**

- 애플리케이션의 확장성과 격리를 향상 시키는 기술
- 애플리케이션과 그에 필요한 종속성을 패키징하여 실행하는 방식
- 격리된 환경 제공 → 다른 컨테이너 및 호스트와 독립적으로 실행
- 빠른 시작 시간 → 일반 가상 머신보다 훨씬 빠르게 실행 가능
- 효율적인 리소스 사용 → CPU, 메모리 등을 최소한으로 활용

- **도커 아키텍처(Docker Architecture)의 구성 요소**

- **도커 클라이언트(Client)**

- 도커 명령어를 실행하는 인터페이스
- 도커 데몬과 통신하여 컨테이너 생성, 삭제, 실행, 관리 수행

- **도커 호스트(Host)**

- 도커 엔진이 설치된 시스템
- 컨테이너를 실행하고 관리하는 역할
- 도커 이미지 저장 및 관리

- **레지스트리(Registry)**

- 도커 이미지를 저장 및 공유하는 중앙 저장소
- 공식 레지스트리: 도커 허브(Docker Hub)
- 개인 레지스트리 운영 가능

- **Docker 활용 및 장점**

- 1. **환경 일관성 유지**

- 개발, 테스트, 운영 환경을 동일하게 유지

- 2. **빠른 배포 및 확장성**

- 컨테이너 단위로 애플리케이션을 배포하여 속도 향상

- 3. **자원 효율적 사용**

- 기존 가상화보다 가볍고 빠르게 실행

4. 격리된 환경 제공

- 컨테이너마다 독립된 실행 공간을 유지하여 보안 강화

5. 빠른 시작 시간

- 필요한 자원만 사용하여 즉시 실행 가능

4.2 Docker 기본 명령어

- 도커 로그인

```
docker login -u <사용자명> # 도커 허브 계정으로 로그인
```

- 이미지 관리

```
docker image pull <이미지명> # 도커 이미지 다운로드
```

```
docker image ls # 다운로드된 이미지 목록 확인
```

```
docker image rmi <이미지명> # 이미지 삭제
```

- 컨테이너 관리

```
docker container run <옵션> <이미지명> # 새 컨테이너 실행
```

```
docker container ps # 실행 중인 컨테이너 목록 확인
```

```
docker container start <컨테이너명> # 컨테이너 시작
```

```
docker container stop <컨테이너명> # 컨테이너 중지
```

```
docker container rm <컨테이너명> # 컨테이너 삭제
```

- 네트워크 설정

```
docker network ls # 현재 사용 중인 네트워크 목록 확인
```

```
docker network create <네트워크명> # 새로운 네트워크 생성
```

- 컨테이너 내부에서 명령 실행

```
docker exec -it <컨테이너명> bash # 컨테이너 내부에서 bash 실행
```

```
docker exec -it <컨테이너명> ls /app # 특정 디렉터리 확인
```

```
docker exec -it <컨테이너명> psql -U user # 데이터베이스 접속
```